Практическая работа 9. Управление событиями пользовательского интерфейса

Пользовательский интерфейс описывает суть **обработки событий** и **паттерн Наблюдатель**. PCMEF-шаблон рекомендует ис-пользовать обработку событий для восходящего уведомления уровней —UNP-принцип. Обработка событий в проекте пользовательского интерфейса касается только одного PCMEF-уровня — уровня `presentation` (представление).

Модель событий библиотеки Swing получается из MVC-шаблона. В Swing аспекты представления и диспетчера MVC объединены в от-дельный компонент (типа `JButton` — кнопка). Для каждого такого компонен-та Swing определяет отдельный интерфейс модели (типа `ButtonModel` — мо- дель кнопки) — рис. 16.7. Все, что должно сделать приложение для изменения предопределенного поведения компонента, это реализовать свой интерфейс модели своим собственным способом.

Любое действие пользователя через доступные ему устройства ввода (мышь, клавиатура и т. д.) вызывает в программе событие, которое должно быть перехвачено объ ектом-источником события — объ ектом **издателя**. Источник события уведомляет о возникновении события каждый объ ект, за-интересованный в этом событии. Эти целевые объ екты являются **объекта-ми-подписчиками** (приемниками, наблюдателями). Каждый источник собы-тия может иметь много подписчиков, и каждый подписчик может слушать много источников события.

Рис. 1 — абстрактный пример обработки события. Каждый класс под-писчика реализует интерфейс приемника библиотеки Swing (типа `ActionListener` — приемник операций) или он расширяет класс, который реализует такой интерфейс. Код, который регистрирует объ ект подписчика в качестве приемника, может быть включен в класс издателя, или эту обязан-ность можно передать отдельному объ екту-регистратору(рис. 9.20).

В примере, когда пользователь нажимает мышью компонент `cancelButton` (кнопка завершения), кнопка вызывает событие действия. Это вызовет метод `actionPerformed()` (выполненное действие) в объекте `Subscriber3` (подписчик 3) — единственный метод интерфейса `ActionListener` библиотеки Swing, который реализован объектом `Subscriber3`. Аргументом для `actionPerformed()` является объект `ActionEvent` (событие действия). `ActionEvent` сообщает объекту `Subscriber3` относительно события и об его источнике (объект `Publisher2` — издатель 2).

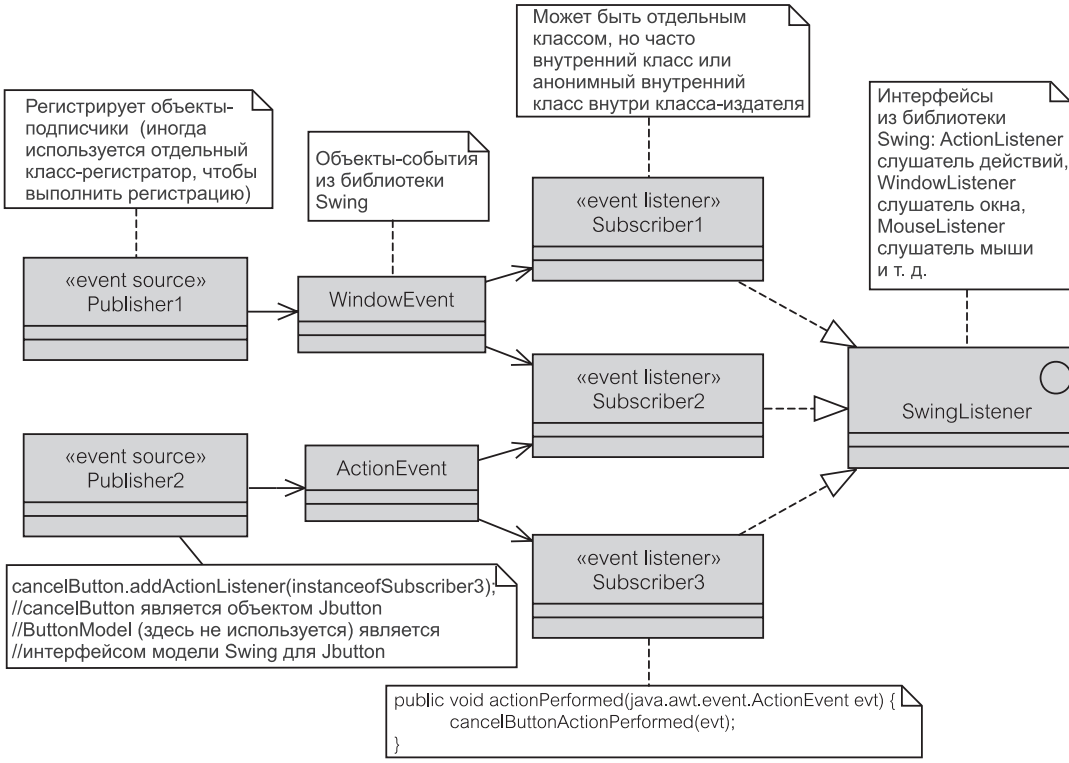


Рис. 1. Управление событиями в пакете Swing

Как отмечено на рис. 1, *класс подписчика* может быть отдельным классом. Это может быть хорошим вариантом в более сложной обработке со-бытий. В PCMEF отдельный класс подписчика, вероятно, будет находиться на уровне `control` (управление) и слушать издателя на уровне `presentation`. В более простых случаях класс подписчика может быть **внутренним классом** класса издателя или даже **анонимным внутренним классом**. Листинги 1, 2 и 3 представляют три примера, которые при-водят к одному и тому же результату: они реализуют `ActionListener` об-ъекта-издателя.

Листинг 1. Подписчик как отдельный класс

Подписчик как отдельный класс

```
public class ShowTitleListener implements ActionListener{
   TitleChanger host; //ссылка на объект, который изменяет
                      заголовок
   public ShowTitleListener(TitleChanger host){
      this.host = host;
```

```
    }
    public void actionPerformed(ActionEvent evt){
                        //запрос изменить поступающий заголовок
       System.out.println("The title is now changed to " +
                                            host.changeTitle());
    }
}
public class TitleChanger{
    public void show(){
       Button b = new Button("Change Title");
       ...
       b.addActionListener(new ShowTitleListener(this));
                    //это то, где данная операция используется
```

Листинг 2. Подписчик как внутренний класс

Подписчик как внутренний класс

```
public class TitleChanger{
    public void show(){
       Button b = new Button("Change Title");
       ...
       b.addActionListener(new ShowTitleListener());
       ...
    }
    //объявление внутреннего класса
    private class ShowTitleListener implements ActionListener{
       public void actionPerformed(ActionEvent evt){
                          //запрос изменить поступающий заголовок
       System.out.println("The title is now changed to " +
       TitleChanger.this.changeTitle());
                                //прямая ссылка на внешний класс
       }
    }
}
```

Листинг 3. Подписчик как анонимный внутренний класс

Подписчик как анонимный внутренний класс

```
public class TitleChanger{
    public void show(){
       Button b = new Button("Change Title");
       ...
       b.addActionListener(new ActionListener(){
```

```
                    //создание анонимного внутреннего класса
          public void actionPerformed(ActionEvent evt){
             System.out.println("The title is now changed to " +
             TitleChanger.this.changeTitle());
                                 //прямая ссылка на внешний класс
          }
       });
       ...
    }
}
```

Все компоненты Swing имеют **интерфейс модели**, определенный в библиотеке. Например, интерфейс модели для кнопок называется `ButtonModel`, для `JList` (класс «список» пакета Swing) он называется `ListModel` (модель списка), для `JTable` (класс «таблица» пакета Swing) он называется `TableModel` (модель таблицы) и т. д. Интерфейсы модели могут быть реализованы для приложения уникальным способом согласно требуемой логике приложения. Однако, для компонентов, не требующих уникальной реализации модели, взаимодействие со стандартной моделью скрыто за методами класса компонента. Это выполняется методом компонента, делегирующим сообщение к его модели, например:

```
public String getTitle() {
  return getModel().getTitle();
}
```

Рис. 2 показывает диаграмму последовательности действий, иллюстрирующую, как **`TableModel`** используется в учебном примере EM. На рисунке интерфейс `TableModel` реализован с помощью `PMessageTableModel` (класс «модель таблицы для сообщений» пакета `presentation`). Рисунок показывает, как `PMessageTableModel` используется для хранения исходящего сообщения и выполнения фильтрации исходящих сообщений, отображаемых объектом `JTable`. Операции сортировки и фильтрации размещены в `PMessageTableModel`. Это сделано потому, что `PMessageTableModel` отвечает за хранение данных, которые могут затем быть отсортированы и отфильтрованы. Запросы фильтрации идут от `PWindow` (класс «окно» пакета `presentation`), как показано на рисунке. `PMessageTableModel` фильтрует данные, делегируя запрос к его `PDisplayList` (класс «список отображения» пакета `presentation`). Последующие операции `JTable`, запрашивающие данные от своей модели (в данном случае от `PMessageTableModel`), приводят к возвращению фильтрованных данных.

Рис. 2 иллюстрирует, как `PWindow` запрашивает `PMessageTableModel` для выполнения фильтрации в соответствии с некоторыми критериями (показа-ны только `datefilter` — фильтрация по дате и `contactfilter` — фильтрация по деловому партнеру). Затем `PMessageTableModel` запрашивает свои данные, `PDisplayList`, чтобы устранить те `PDisplayData` (класс «отображаемые данные» пакета `presentation`), которые не удовлетворяют критерию. Как только фильтрация будет закончена, `PWindow` вызывает экран, который бу-

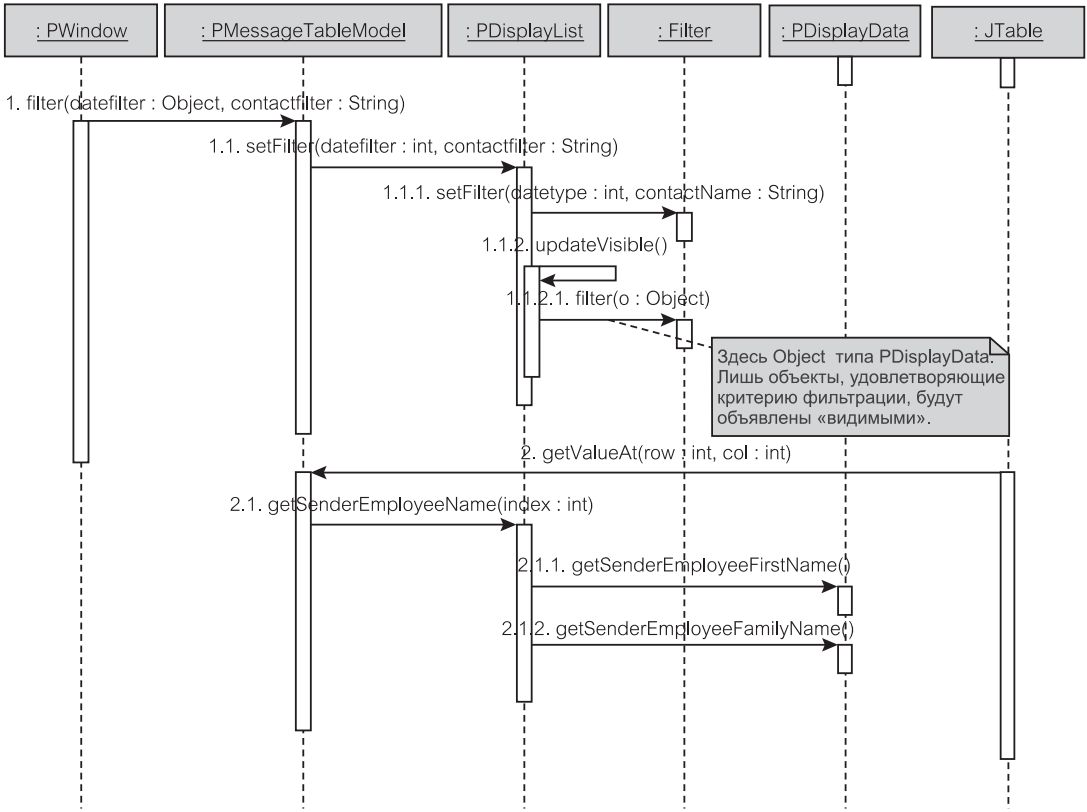


Рис. 2. Диаграмма последовательности действий, включающая PMessageTableModel

дет перерисован, и заставляет `JTable` показать измененное подмножество данных. `JTable` запрашивает свою модель, `PMessageTableModel`, для получения *нового* списка сообщений, которые будут отображены на экране.

`JTable` запрашивает данные на основе строк и столбцов. `PMessageTableModel` переводит этот запрос на соответствующие поля, размещенные в модели. Эта обработка выполняется сообщением `getSenderEmployeeName()` (получить имя служащего-отправителя) к `PDisplayList`. `PDisplayList` затем формирует имя служащего-отправителя, объединяя его/ее имя и фамилию.

9.1. Паттерны и пользовательский интерфейс

Графические пользовательские интерфейсы (Graphical user interfaces — GUI) в значительной степени ответственны за переключение от процедурного к объектно-ориентированному ПО. Хотя объектно-ориентированное ПО и языки программирования были известны с 1960-х годов, использование GUI требовало перехода к использующему события программированию, свойствен-

ному объектной ориентации. GUI перемещают центр управления от программы к пользователю. Пользователь отвечает за выполнение программы, решая, какие действия должны быть далее выполнены. Эти действия переводятся во входные события. Каждое входное событие требует объекта, который будет обслуживать его.

Графические пользовательские интерфейсы также в значительной степени ответственны за популярность **паттернов** в проектировании системы. «Вид и поведение» обеспечивают продажу ПО. Пользователи требуют ранних прототипов пользовательского интерфейса, чтобы оценить вид и поведение системы. Интегрированные среды разработки (Integrated development environments — IDE) и технологии быстрой разработки приложения (Rapid Application Development — RAD) позволяют быстро создавать такие прототипы. Все эти требования приводят к широкому повторному использованию компонентов GUI. Такой прогресс зарегистрирован в паттернах, относящихся к GUI, и наоборот, большинство особо важных паттернов **Банды четырех** (Gang of Four — GoF) может быть найдено в проекте компонентов GUI.

Этот раздел охватывает только небольшое подмножество обычно используемых паттернов в разработке пользовательского интерфейса. Рассматриваемые паттерны следующие: Наблюдатель (Observer), Декоратор (Decorator), Цепочка обязанностей (Chain of Responsibility) и Команда (Command). Паттерны Наблюдатель и Цепочка обязанностей были обсуждены ранее в других контекстах.

9.1.1. Наблюдатель

Различные компоненты в пользовательском интерфейсе наблюдают объекты модели (сущности) согласно шаблону Model-View-Controller. Они изменяют поведение или представление пользовательского интерфейса, основанные на полученных уведомлениях о том, что модель изменилась. Та-кое отношение наблюдения модели удобно назвать **паттерном Наблюдатель**.

Рассмотрим пример отношения между компонентом `JTable` (класс «таблица» пакета Swing) и его моделью `JTableModel` (класс «модель таблицы» пакета Swing), как показано на рис. 3. Таблица регистрирует события своей модели. После модификации модели таблица уведомляется об этом и поэтому способна соответственно ответить. Рис. 3 показывает наблюда-телей, реализованных на взаимодействии между `JTable` и его моделью. Рисунок не включает все компоненты, связанные с `JTable`, так как их доста-точно много. Он показывает только `JTableModel` и `TableCellEditor` (редактор ячеек таблицы).

Таблица сформирована с экземпляром `JTableModel`. Она регистрирует свой интерес к модели через метод `addTableModelListener()` (добавить приемник модели таблицы). Это гарантирует, что таблица будет уведомлена, когда модель изменится.

Когда пользователь решает изменить ячейку в таблице двойным нажатием мышью по ячейке, таблица строит `CellEditor` (редактор ячейки). В `CellEditor` (или скорее его редактируемом компоненте) устанавливается величина, извлеченная из модели через `getValueAt()` — получить значе-

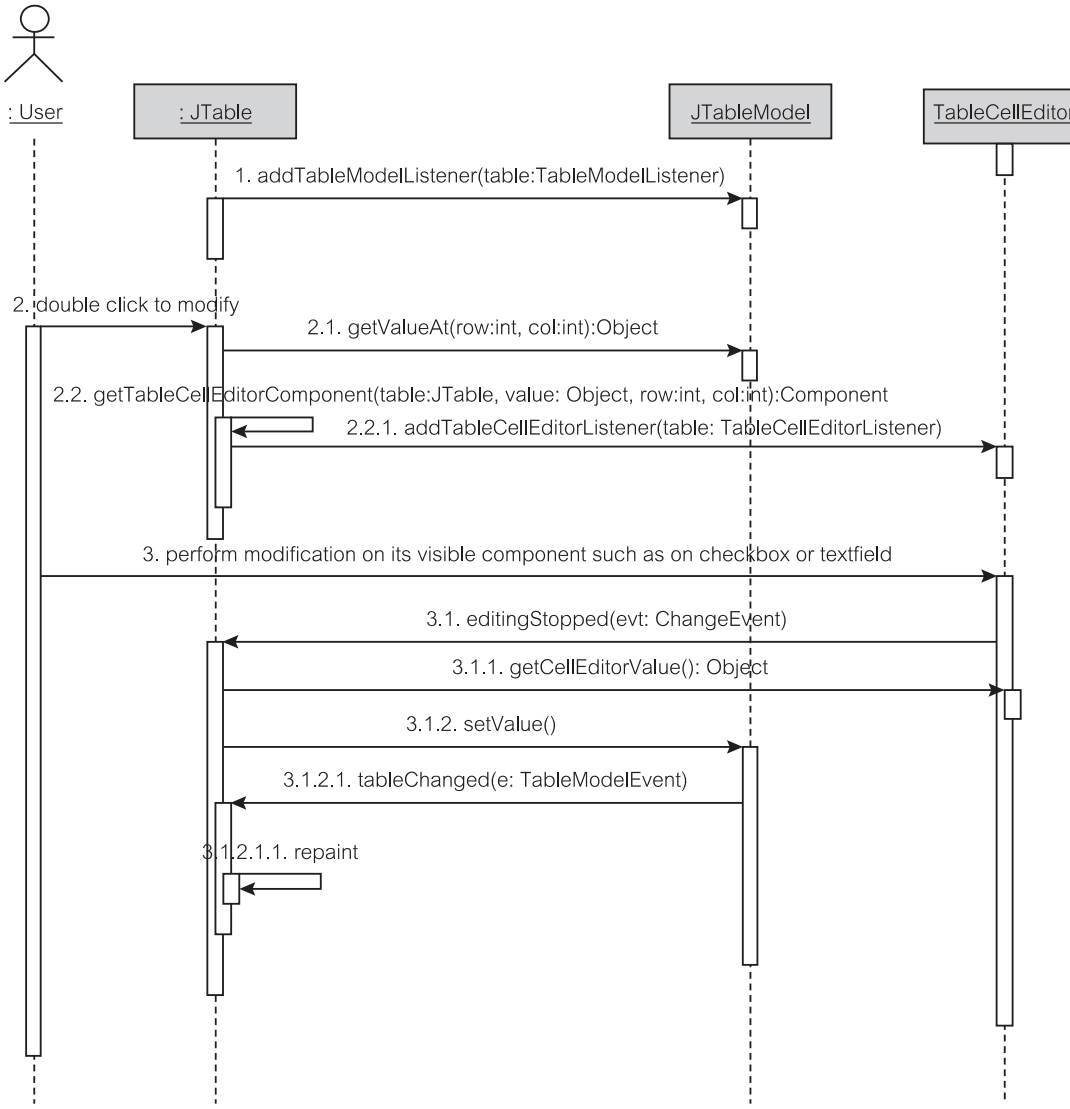


Рис. 3. Наблюдатели

ние о... (сообщение 2.1). Что интересно отметить, так это то, что сам `TableCellEditor` фактически является моделью по отношению к `JTable`. `JTable` — подписчик (приемник) к `TableCellEditor` (сообщение 2.2.1).

Рис. 3 использует паттерн Наблюдатель для:

- наблюдения компонентом `JTable` модели `JTableModel` (сообщение 1);
- наблюдения компонентом `JTable` редактора `TableCellEditor` (сообщение 2.2.1).

Когда на редактируемом компоненте типа выключателя или текстового поля выполнена модификация (сообщение 3), `TableCellEditor` уведомляет таблицу через ее метод `editingStopped()` — редактирование завершено (реализация Наблюдателя из сообщения 2.2.1). В этом случае таблица может обработать измененную величину, извлеченную из `getCellEditorValue()` (получить значение редактора ячейки), и изменить значение ее модели `JTableModel` (сообщение 3.1.2 `setValue()` — задать значение).

`JTable` действует как посредник между величиной, измененной в компоненте `TableCellEditor`, и величиной, запасенной в `JTableModel`. Модификация в `JTableModel` вызывает уведомление всех заинтересованных сторон (`JTable` и другие классы, которые регистрируются через `addTableModelListener()`, сообщение 1). Уведомление осуществляется вызовом их метода `tableChanged()` (таблица изменена). Наконец наблюдатель `JTable`, перерисовывает себя (сообщение 3.1.2.1.1), запрашивая модель о ее данных, которые будут показаны на экране.

Паттерн Наблюдатель в проекте пользовательского интерфейса используется широко. Это позволяет разрабатывать объединения компонентов пользовательского интерфейса и взаимодействия компонентов, обладающие большими возможностями (сложные уведомления через события). Паттерн Наблюдатель имеет дело с требованиями систем, управляемых событиями, так что модель может быть изменена в любое время, и пользовательский интерфейс должен знать о таких модификациях.

9.1.2. Декоратор

Паттерн Декоратор позволяет использовать ряд классов-вместилищ, чтобы *украсить* внешний вид компонента. Это происходит, например, когда используются полосы прокрутки, чтобы украсить `JList` (класс «список» пакета Swing), формируя прокручиваемый список. Другой пример — `Frame` (структура), украшенная полосами прокрутки, строкой меню, панелью инструментов и границами.

Рис. 1 иллюстрирует использование декораторов. Он показывает `Panel` (панель), украшенную границей с названием «`ButtonGroup`» (группа кнопок). Он также показывает `JList`, помещенный внутри `JScrollPane` (класс «прокручиваемая панель» пакета Swing). `JScrollPane` является прокручиваемой панелью.

Паттерн Декоратор используется, чтобы украсить и увеличить привлекательность (дружелюбие к пользователю) пользовательского интерфейса. Единственной целью некоторых компонентов пользовательского интерфейса, типа различных границ, является использование их в качестве декораторов.

9.1.3. Цепочка обязанностей

Когда компонент неспособен обеспечить обслуживание отдельного запроса, он может решить делегировать такую обязанность своим подкомпонентам, пока один из подкомпонентов не сможет обслужить запрос. Связанный с этим паттерн называется **Цепочкой обязанностей**.

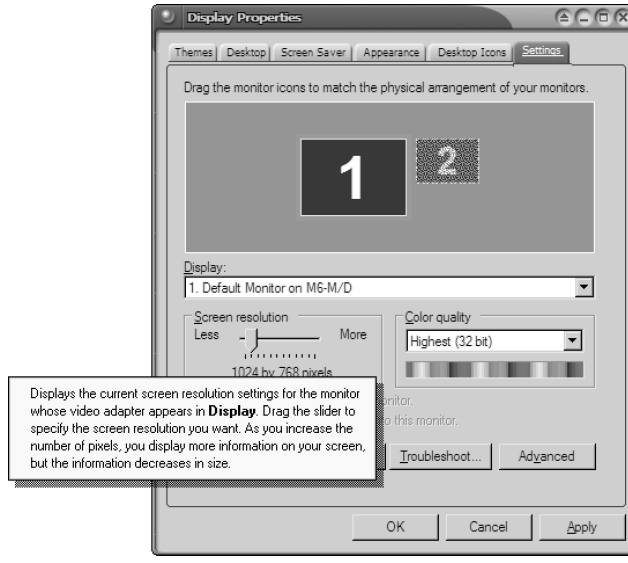


Рис. 4. Help (помощь): Цепочка обязанностей

ОС Windows реализует этот паттерн, чтобы поддержать свою систему *Help* (помощь). Всякий раз, когда пользователь Windows не понимает цели некоторого элемента управления в GUI, он/она может нажать правой клавишей по компоненту, чтобы найти разъяснение его цели. Рис. 4 показыва-ет, как текущий компонент (ползунок для изменения разрешения экрана) от-вечает на нажатие правой клавишей пользователем и выбор элемента меню «What's this?» (Что это?).

Текущий компонент определит, может ли он ответить на запрос. Если нет, он делегирует запрос к следующему объекту в цепочке обязанностей. В конечном счете должен быть найден некий компонент, который может отобразить соответствующую информацию на экране и остановить *цепочку*.

На рис. 4 текущий компонент делегирует запрос к своему контейнер-ному компоненту. Контейнерный компонент — панель с границей, содержа-щая название «`Screen Resolution`» (разрешение экрана). Если панель имеет ответ, она отобразит информацию на экран и остановит дальнейшую передачу запроса. Цепочка запросов продолжается до самого верхнего кон-тейнерного компонента, который может или представить информацию (это может быть или конкретная, или общая информация) или остановить цепочку без ответа.

Этот паттерн упрощает проектирование обычно требуемых последовательностей делегирования, типа системы *Help*. Однако у этого паттерна имеется недостаток. Он требует, чтобы все вложенные контейнеры реализовали этот паттерн. Контейнер или компонент должны быть способны решать, может ли он ответить на запрос или должен передать его далее. Кроме того, компоненты увеличиваются в размере, если они хотят участвовать в нескольких различных системах контекстной *помощи*, то есть больше чем в одном контексте цепочек обязанностей.

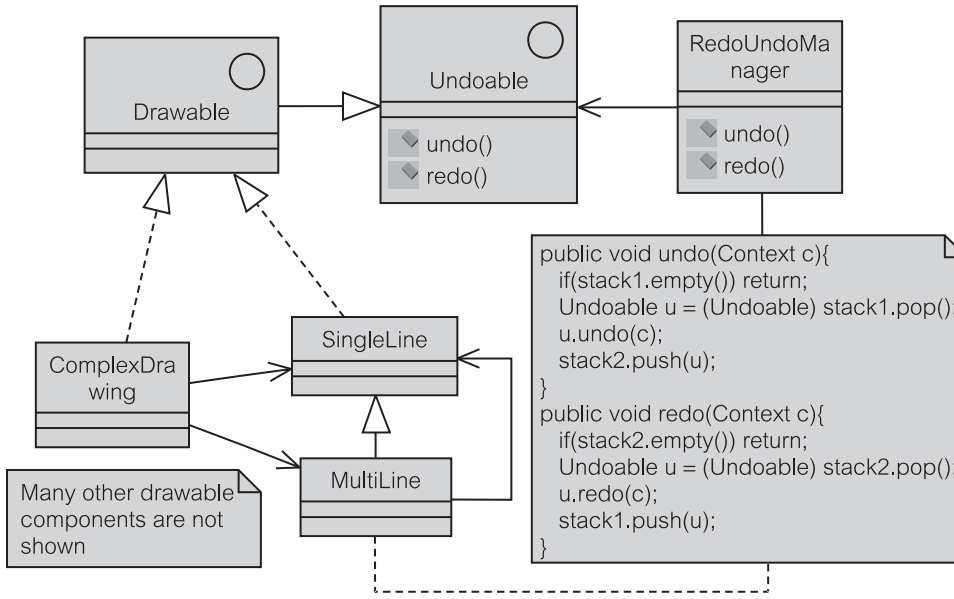


Рис. 5. Пример паттерна Команда: Undo (отмена)

9.1.4. Команда

Паттерн Команда подобен паттерну Цепочка обязанностей. Он делеги-рует запрос к следующему компоненту, пока один из них не сможет обслу-жить запрос. Небольшое отличие от Цепочки обязанностей, где запрос делегируется родительскому контейнеру и может идти до вершины иерархии, заключается в том, что компоненты в паттерне Команда делегируют запрос известному интерфейсу.

Рассмотрим, например, выполнение приложения. Приложение нужно обеспечить механизмом отмены/восстановления. Каждая операция, выполненная в приложении, должна быть *зарегистрирована*, чтобы позволить приложению возвратиться к своим предыдущим состояниям. Однако операции, выполняемые в приложении, настолько разнообразны, что имеют различные средства *отмены*. Например, перемещение прямоугольника можно рассматривать как размещение четырех строк, каждая в одну линию. Паттерн Команда требует, чтобы независимо от этого разнообразия, вложенные компоненты реализовывали простой интерфейс, и поэтому с ними можно было бы обращаться единообразно.

Рис. 5 показывает проект отмены, который следует паттерну Команда. `RedoUndoManager` (менеджер восстановления/отмены) не знает, как каждый из его компонентов, размещенных в стеке, выполняет действия отмены или восстановления. Когда требуется отмена, он восстанавливает прежнюю величину из `stack1` (стек 1) и вызывает свой метод `undo()` (отменить). Сама отмена может быть достаточно сложной операцией, но она оставлена для выполнения компонентом.

Сложный компонент, вроде класса `MultiLine` (несколько строк), может далее использовать свойство `Undoable` (допускает отмену), чтобы выполнить свой метод `undo()`, как показано на рис. 5.

9.2. Пользовательский интерфейс для управления электронной почтой

В итерации 2 пользовательские интерфейсы не сильно отличаются от прототипа. Имеются незначительные изменения из-за уда-ления или добавления некоторых элементов управления окном, чтобы сде-лать понятным и упростить GUI. Например, на рис. 6 был изменен текст на командных кнопках по сравнению с его прототипом на рис. 3. Ярлыки кнопок с единственным словом типа *Insert* (добавить) заменяют более длин-ный текст, предложенный в прототипе, типа *Insert New Message* (добавить новое сообщение). Добавлена очень важная кнопка *Exit* (выход).

Отображение исходящих сообщений на рис. 6 сортируется в убываю-щем порядке параметра `Scheduled Dtp` (намеченный отдел). Нажатие кла-вишей мыши по столбцу, содержащему имена, выполняет сортировку. При первоначальном отображении исходящих сообщений опции фильтрации по умолчанию запрещены. Нажатие мышью кнопки `Toggle Filter` (переключатель фильтра) делает нижерасположенные опции доступными.

Отображаются дополнительные сообщения, чтобы информировать пользователя относительно неполной информации, представленной в GUI, типа показанной на рис. 7. Пример ссылается на подпоток *Create New Message* (создать новое сообщение), представленный в главе 14 на рис. 14.6.

Добавление исходящего сообщения, использующее диалоговое окно, показанное на рис. 14.6, вызывает ряд интересных действий. Прежде чем диа-

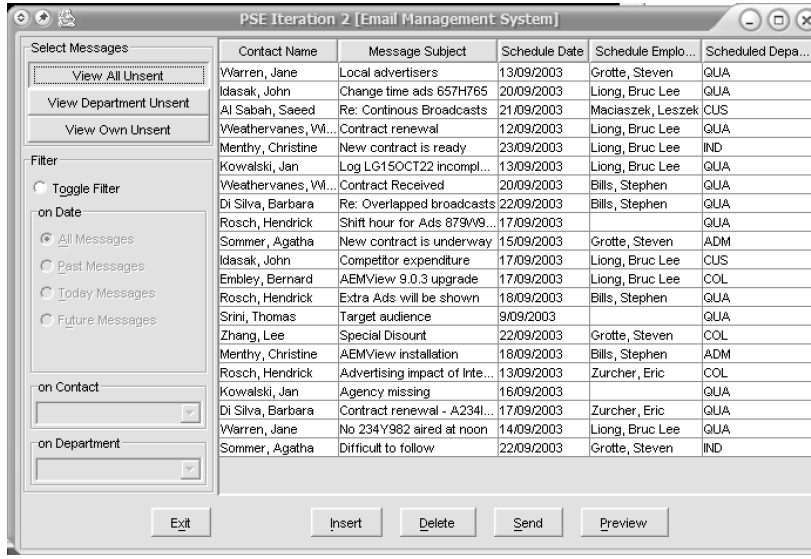


Рис. 6. Итерация 2: основной GUI

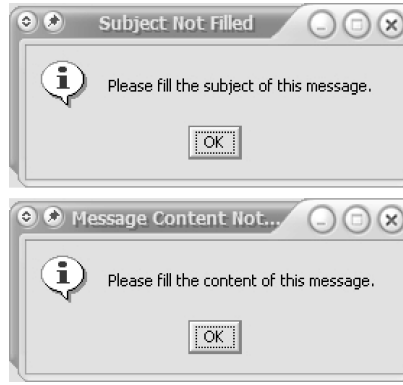


Рис. 7. Сообщения об ошибках для Create New Message (создать новое сообщение)

логовое окно можно показать на экране, программа должна построить заместителя исходящего сообщения и гарантировать, что исходящее сообщение зарегистрировано в `EIdentityMap` (класс «коллекция идентичности объектов» пакета `entity`). Рис. 8 показывает, что создание OID (идентификатор объекта) для нового исходящего сообщения обработано с помощью `MDataMapper` (класс «преобразователь данных» пакета `mediator`).

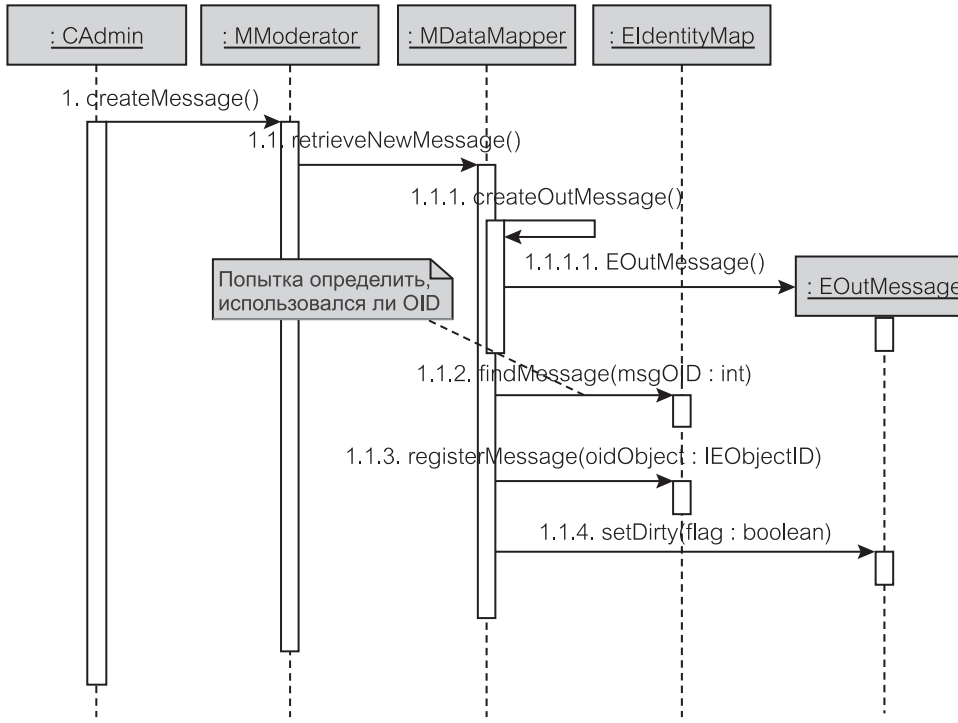


Рис. 8. Получение заместителя нового исходящего сообщения

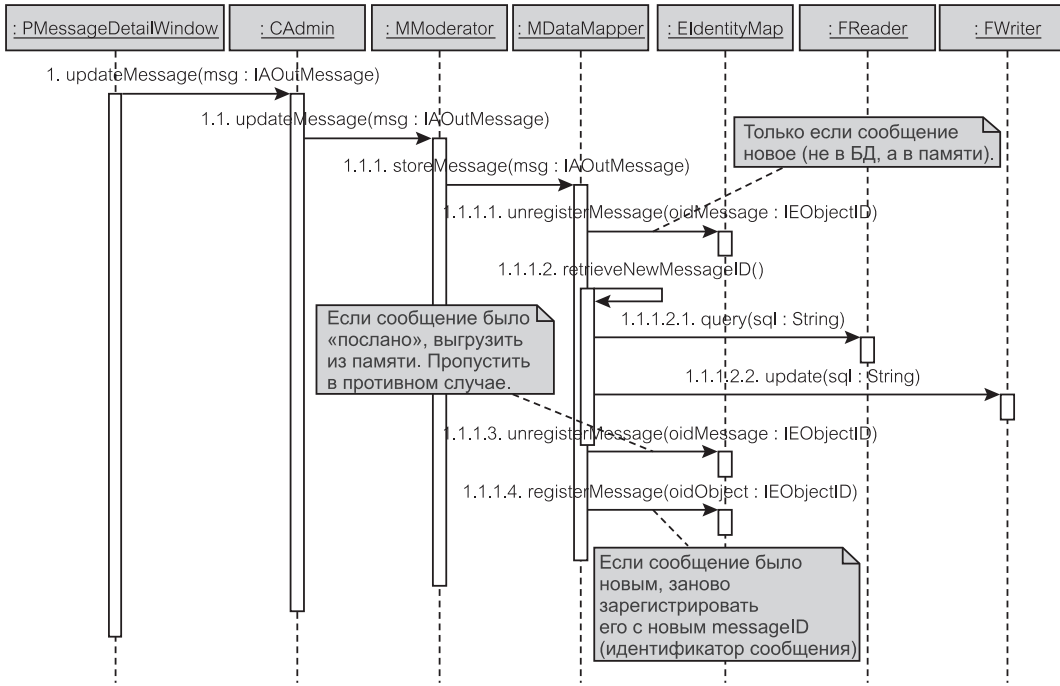


Рис. 9. Корректировка модифицированного исходящего сообщения

Далее новое исходящее сообщение автоматически помечается как измененное, чтобы указать, что его требуется включить в следующий цикл корректировки (или перезагрузки), выполняемый с помощью `EIdentityMap`. Как только заместитель исходящего сообщения получен, исходящее сообщение может быть изменено и обновлено с помощью `PMessageDetailWindow` — класс «окно деталей сообщения» пакета `presentation`.

Корректировка нового созданного исходящего сообщения происходит в конце выполнения подпотока *S5 — Create New Message* — создать новое сообщение. Рис. 9 показывает, что механизм сохранения совмес-тим с паттернами *Коллекция идентичности объектов* (Identity Map) и *Преоб-разователь данных* (Data Mapper).

Рис. 9 иллюстрирует сценарий, где пользователь закончил редактиро-вать исходящее сообщение в `PMessageDetailWindow` и хочет сохранить его в БД. Сохранение исходящего сообщения требует, чтобы `MDataMapper` синхронизировал содержание кэша с содержанием БД (поскольку данное исходящее сообщение имеет к этому отношение). В первую очередь, `MDataMapper` не должен регистрировать исходящее сообщение из `EIdentityMap`, если это сообщение только что было создано, то есть оно существует только в памяти, но его еще нет в БД.

Не выполнять эту регистрацию следует потому, что `MDataMapper` первоначально зарегистрировал исходящее сообщение в `EIdentityMap`, исполь-

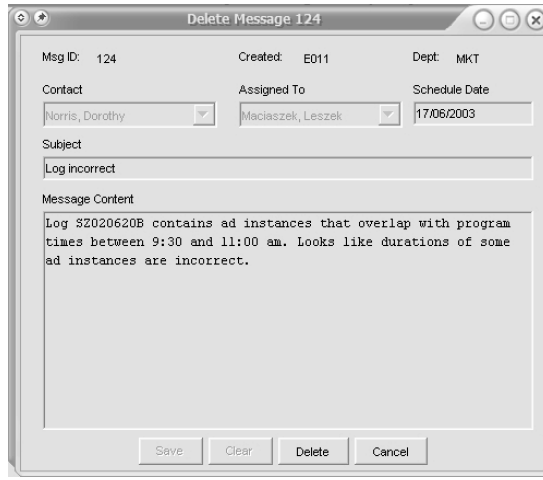


Рис. 10. Окно для удаления исходящего сообщения

зуя автоматически формируемый OID, а не определенный на основе свойств реального исходящего сообщения. Если исходящее сообщение не зарегистрировано, его соответствующий `messageID` (идентификатор сообщения) получается из БД до того, как это сообщение будет сохранено в БД. Получение `messageID` для нового исходящего сообщения завершает всю совокупность требуемых данных, необходимых для `EOutMessage`. Корректировка только что созданного сообщения может теперь быть завершена перерегистрацией в `EIdentityMap` со своими OID и `messageID`.

Другие изменения по сравнению с прототипом GUI, касаются использования соответствующего диалога, чтобы подтвер-дить, что пользователь хочет удалить сообщение. Диалоговое окно, показан-ное на рис. 10 теперь заменяет окно, изображенное на рис. 7.

С этим новым окном пользователь информирован, какое исходящее сообщение предполагается удалить, и поэтому можно избежать ошибок. Поля в диалоговом окне не редактируются.

Имеются другие незначительные усовершенствования в проекте пользовательского интерфейса для итерации 2. Например, сообщение об ошибке, чтобы показать, что отдельные исходящие сообщения не были посланы, теперь иллюстрируется на рис. 11.



Рис. 11. Ошибка «Unable to Send» (невозможно послать)

Резюме

1. Проект пользовательского интерфейса (User interface — UI) должен учитывать две основные категории клиентов: программируемый клиент и клиент-браузер. *Программируемый клиент* предполагает, что программа находится и выполняется на клиенте и имеет доступ к ресурсам, хранящимся на клиенте. *Клиент-браузер* нуждается в сервере, чтобы загрузить требуемые данные и получить инструкции для предоставления данных в пользовательском интерфейсе на основе Web-технологии.
2. Кроме предоставления функциональных возможностей системы пользователю, пользовательский интерфейс должен удовлетворять ряду других требований, известных под термином *применимость*. Основные руководящие принципы проектирования пользовательского интерфейса: 1) пользователь в управлении, 2) непротиворечивость интерфейса, 3) снисходительность интерфейса и 4) адаптируемость интерфейса.
3. *Библиотеки классов* пользовательского интерфейса содержат «оконные» компоненты и определяют впечатление и ощущение от использования приложения. В мире Java библиотека классов называется Java™ Foundation Classes (JFC). Большая часть JFC состоит из набора GUI-компонентов по имени Swing. Компоненты Swing могут быть сгруппированы в контейнеры, меню и элементы управления.
4. *Контейнеры* определяют основные вид и поведение приложения. Они представляют прямоугольные области на рабочем столе пользовательского интерфейса и содержат другие компоненты, включая другие контейнеры, меню и элементы управления. Окна, диалоговые окна, части окон и панели — примеры контейнеров. Swing использует «менеджер расположения», чтобы разместить компоненты внутри контейнера.
5. Окна и некоторые другие компоненты могут содержать *меню*. Компоненты меню используются, чтобы реализовать строки меню и их элементы. Выключатели и кнопки-переключатели могут рассматриваться как виды элементов меню. Панели инструментов могут также рассматриваться как меню.
6. *Элементы управления* представляют модель событий пользовательского интерфейса. Они делятся на командные кнопки и другие элементы управления. Обычные кнопки, кнопки-переключатели, выключатели и элементы меню — все они являются разновидностями командных кнопок. Другие элементы управления включают окна со списками элементов для выбора нескольких или только одного элемента (комбинированная строка ввода), полосы прокрутки и ползунки. Один из наиболее универсальных и полезных элементов управления — представление дерева контейнеров и элементов.
7. *Модель событий* библиотеки Swing получена из MVC-шаблона. Обработка пользовательским интерфейсом событий должна соответствовать паттерну Наблюдатель.
8. Обычно используемые паттерны в разработке пользовательского интерфейса: Наблюдатель, Декоратор, Цепочка обязанностей и Команда.

Контрольные вопросы:

1. Какую роль играет обработка событий в PCMEF-шаблоне с точки зрения принципа восходящего уведомления уровней (UNP) и какого уровня архитектуры она касается?
2. В чем заключается взаимодействие между объектом-источником события (издателем) и объектами-подписчиками (приемниками), и как происходит регистрация подписчика на примере интерфейса **ActionListener**?
3. Как организованы сортировка и фильтрация данных в компоненте **JTable** на примере реализации модели **PMessageTableModel** и классов **PDisplayList** и **PDisplayData**?
4. Как паттерн «Наблюдатель» (**Observer**) реализуется при взаимодействии компонента **JTable** с его моделью **JTableModel** и редактором ячеек **TableCellEditor**?
5. Как работает паттерн «Цепочка обязанностей» **(Chain of Responsibility)** на примере контекстной справочной системы **Windows** («What's this?») и каковы его основные недостатки?
6. В чем заключается главное отличие паттерна «Команда» (**Command**) от паттерна «Цепочка обязанностей» и как с его помощью реализуется механизм отмены и восстановления действий (**Undo/Redo**) на основе стеков?
7. Какова последовательность действий при создании заместителя нового исходящего сообщения до открытия окна редактирования, и какую роль в этом процессе играют **MDataMapper**, **OID** и **EIdentityMap**?
8. Как осуществляется синхронизация кэша и базы данных при сохранении нового или отредактированного исходящего сообщения из окна **PMessageDetailWindow**?
9. В чем заключаются основные различия между двумя категориями клиентов при проектировании пользовательского интерфейса: программируемым клиентом и клиентом-браузером?
10. На какие три основные группы делятся GUI-компоненты библиотеки **Swing** (в составе JFC) и какую функцию выполняют менеджеры расположения?